Šepet

Varno sporočanje s šifriranjem od konca do konca

Prijava teme projekta · Računalniške storitve v oblaku 2026/27 · UL FRI

Projektna ekipa 13 »Ključarji« · Tibor Koderman, Amar Mehić, Eva Vidic · 26. 10. 2026

1 Osnovni podatki

Naslov projekta	Šepet — aplikacija za varno sporočanje s šifriranjem od konca do konca
Projektna skupina	Projektna ekipa 13
Ime skupine	Ključarji
Člani	Tibor Koderman, Amar Mehić, Eva Vidic
Repozitoriji	https://gitlab.koderverse.com/fri-mag/rso — javna skupina GitLab z enim projektom za vsako mikrostoritev (ime in opis projekta sta ime in opis mikrostoritve) ter projekti za aplikacijo, funkcije, platformo, skupne predloge CI in dokumentacijo
Oblak	Microsoft Azure (Azure for Students): Azure Kubernetes Service, Blob Storage, Functions, Key Vault

Vsakdo v ekipi prevzame lastništvo vsaj dveh mikrostoritev in nekaj prečnih področij: skrbi za kodo, teste, Helm chart in svoj del poročila, preostala člana pa opravita pregled kode.

Član	Mikrostoritve	Prečna področja
Tibor Koderman	identity-service (ponudnik identitet), keys-service	platforma (AKS, Terraform, Helm, GitLab CI), prehod API, zasnova šifriranja
Amar Mehić	conversation-service, message-service, realtime-service	NATS JetStream, Event Sourcing in CQRS, ScyllaDB, gRPC, GraphQL
Eva Vidic	media-service (+ media-functions), notification-service	aplikacija Flutter z integracijo MLS, Azure Functions, Firebase Cloud Messaging

2 Kratek opis projekta

Velik del zasebne komunikacije teče prek storitev, ki jim moramo verjeti na besedo: Telegram privzetih in skupinskih klepetov ne šifrira od konca do konca[1], ponudniki pa so pod pritiskom, naj oblastem omogočijo dostop do vsebine — od evropskega predloga uredbe CSAR (»Chat Control«), o katerem pogajanja še tečejo[2], do britanske zahteve, zaradi katere Apple od februarja 2025 novim uporabnikom v Združenem kraljestvu ne ponuja več napredne zaščite podatkov[3]. Takšne dostope lahko zlorabijo tudi napadalci, kot je pokazal vdor Salt Typhoon v ameriška telekomunikacijska omrežja (2024), po katerem je agencija CISA ogroženim uporabnikom priporočila komunikacijo, šifrirano od konca do konca[4]. **Šepet** je aplikacija za zasebno sporočanje ena na ena in v skupinah, z oblikovanim besedilom in slikami, v kateri se vsebina šifrira in dešifrira izključno na napravah udeležencev s standardiziranim protokolom MLS (RFC 9420)[5]; strežnik hrani in posreduje le šifrirane ovojnice in nujne metapodatke, zato vsebine ne more prebrati niti je predati komurkoli. Cilj projekta je pokazati, da je tak sistem mogoče zgraditi kot skalabilno, odporno in opazljivo domorodno oblačno (*cloud-native*) aplikacijo — sedem mikrostoritev v .NET 10 z bazama ScyllaDB in PostgreSQL, dogodki v NATS JetStream, lastnim

[1] Telegram FAQ (*cloud chats* uporabljajo šifriranje strežnik–odjemalec, *secret chats* so samo ena na ena in jih je treba ročno vklopiti): https://telegram.org/faq, https://core.telegram.org/api/end-to-end

[2] Evropska komisija, predlog uredbe COM(2022) 209, ki je predvidel odredbe o odkrivanju, s katerimi bi lahko ponudnikom naložili pregledovanje zasebnih sporočil: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=COM:2022:209:FIN; stališče Sveta (26. 11. 2025) brez obveznih odredb o odkrivanju: https://www.consilium.europa.eu/en/press/press-releases/2025/11/26/child-sexual-abuse-council-reaches-position-on-law-protecting-children-from-online-abuse/

[3] Apple, »Advanced Data Protection is no longer available for new users in the UK« (februar 2025): https://support.apple.com/122234

[4] FBI in CISA, skupna izjava, 13. 11. 2024: https://www.cisa.gov/news-events/news/joint-statement-fbi-and-cisa-peoples-republic-china-prc-targeting-commercial-telecommunications; CISA, *Mobile Communications Best Practice Guidance*, 18. 12. 2024: https://www.cisa.gov/sites/default/files/2024-12/guidance-mobile-communications-best-practices.pdf

[5] R. Barnes et al., *The Messaging Layer Security (MLS) Protocol*, RFC 9420, julij 2023; arhitektura: RFC 9750, april 2025.

ponudnikom identitet in odjemalcem v Flutterju, nameščenih v Azure Kubernetes Service — ne da bi strežnik kdajkoli potreboval ključe za dešifriranje.

Varnostni model — kaj strežnik vidi in česa ne:

- **Vsebina:** sporočila, slike ter imena in slike skupin se šifrirajo na napravi; priponke se šifrirajo z naključnim ključem (AES-256-GCM), ki potuje v šifriranem sporočilu.
- **Ključi:** zasebni ključi ostanejo na napravi; strežnik hrani le javne ključe in enkratne pakete ključev (MLS KeyPackage).
- **Podtaknjeni ključi:** strežnik bi lahko poskusil podtakniti lažen ključ, zato uporabnika lahko primerjata varnostno številko (koda QR), keys-service pa vodi dnevnik sprememb ključev, verižen z zgoščenimi vrednostmi, ki ga aplikacija preverja.
- **Naprejšnja tajnost in varnost po kompromitaciji:** ob vsaki spremembi članstva skupina preide v novo epoho, zato odstranjen član novih sporočil ne more dešifrirati; naprave redno osvežujejo svoje ključe (MLS Update in Commit).
- **Metapodatki in omejitve:** strežnik vidi članstvo v pogovorih (potrebno za usmerjanje) ter čas in zapolnjeno velikost ovojnic; potisna obvestila, dnevniki in metrike ne vsebujejo vsebine. Zunaj obsega sta kompromitirana naprava in skrivanje metapodatkov (*sealed sender*).

3 Ogrodje in razvojno okolje

Področje	Izbira	Zakaj
Uporabniški vmesnik	**Flutter** (Dart) za Android in splet; Riverpod, go_router, dio	ena koda za mobilno in spletno aplikacijo; spletna različica je javno dostopna za demonstracijo
Šifriranje E2E	**MLS (RFC 9420)** prek knjižnice **OpenMLS** (Rust, MIT; Dart paket `openmls`) za vmesnikom `CryptoEngine`; lokalna zbirka SQLite + SQLCipher	standard IETF za skupine; 1:1 je skupina dveh, naprava je član. Zaledje hrani neprozorne pakete ključev in ovojnice, zato je preklop na rezervo — Signal Protocol (`libsignal_protocol_dart`) — omejen na aplikacijo in manjše prilagoditve dveh storitev
Mikrostoritve	**.NET 10 (LTS)**, ASP.NET Core Minimal API, Clean Architecture	zrelo ogrodje z vgrajenim OpenAPI 3.1, gRPC, preverjanjem zdravja in OpenTelemetry; en jezik za vse storitve olajša skupno predlogo
Knjižnice zaledja	ASP.NET Core Identity + OpenIddict (OAuth 2.0/OIDC), Marten (Event Sourcing na PostgreSQL), Hot Chocolate (GraphQL), Grpc.AspNetCore, ScyllaDB C# Driver, NATS.Net, Microsoft.Extensions.Http.Resilience (Polly v8), EF Core + Npgsql	vsaka pokriva eno zahtevo predmeta s preverjeno rešitvijo, brez lastne kriptografije
Podatkovne baze	**ScyllaDB** (sporočila), **PostgreSQL** (ostale storitve s trajnimi podatki), Azure Blob Storage (šifrirane priponke)	ScyllaDB za časovno urejen dnevnik sporočil z veliko zapisovanja in vgrajenim TTL — Discord na njej hrani bilijone sporočil[6]; PostgreSQL za transakcijske podatke, identitete in shrambo dogodkov
Sporočila med storitvami	**NATS JetStream** (+ NATS KV za prisotnost)	lahek posrednik s trajnimi tokovi, ponovnim branjem in dostavo vsaj enkrat
Identiteta	**lasten ponudnik identitet v .NET** (identity-service: OAuth 2.0, OpenID Connect, PostgreSQL)	veže uporabnike na kriptografske identitete naprav (storitev za avtentikacijo v arhitekturi MLS); noben tretji ponudnik ne izve, kdo uporablja aplikacijo
Platforma	Docker, Kubernetes (AKS 1.36), Helm, **Envoy Gateway** (Gateway API), cert-manager, KEDA, CloudNativePG, Scylla Operator	prehod z JWT (JWKS), omejevanjem hitrosti in TLS; Gateway API nadomešča upokojeni ingress-nginx

[6] Discord, *How Discord Stores Trillions of Messages*, 2023: https://discord.com/blog/how-discord-stores-trillions-of-messages

Področje	Izbira	Zakaj
CI/CD in IaC	**GitLab CI** (skupne predloge v projektu `ci-templates`, OIDC do Azure), register vsebnikov GitLab, skeniranje odvisnosti in slik (Trivy), SBOM (Syft); **Terraform** (azurerm)	ista slika v vseh okoljih: samodejna namestitev v okolje *dev* (veja `dev`) in *prod* (veja `main`, z ročno odobritvijo); projekte lahko zrcalimo na GitHub
Opazovanje	OpenTelemetry, Prometheus, Grafana, Fluent Bit + Loki, Alertmanager; k6 za obremenitvene teste	metrike RED/USE, centralizirani dnevniki in opozorila na vzorce v dnevnikih
Brezstrežniško	**Azure Functions** (.NET 10 isolated, Flex Consumption), Event Grid, Application Insights	dogodkovno preverjanje in čiščenje priponk, ki se samodejno skalira
Zunanji API	**Firebase Cloud Messaging HTTP v1** (OAuth 2.0 s storitvenim računom)	potisna obvestila brez vsebine
Pogodbe	OpenAPI 3.1 (+ Scalar), AsyncAPI 3.0 + CloudEvents 1.0, Protobuf, GraphQL SDL, napake po RFC 9457	pogodba je v repozitoriju storitve, preden napišemo kodo
Razvojno okolje	JetBrains Rider ali VS Code (C#), Android Studio ali VS Code (Flutter), Docker Compose in kind za lokalno gručo, Git (veji `main` in `dev`, zahtevki za združitev s pregledom)	lokalno okolje z enim ukazom; v oblak gre samo preverjena koda

4 Shema arhitekture

Slika 1 prikazuje vsebnike (raven 2 modela C4) in protokole med njimi. Komunikacijo izbiramo po pravilu: če klicatelj potrebuje rezultat, da lahko nadaljuje, kliče sinhrono (REST ali gRPC, s časovno omejitvijo in nadomestnim odzivom); če gre za dejstvo, ki zanima druge storitve, objavi dogodek v NATS JetStream.

Interakcija	Protokol	Zakaj tako
aplikacija → storitve (ukazi, poizvedbe)	REST prek prehoda API	javni API z OpenAPI; `Idempotency-Key` pri pošiljanju, napake RFC 9457
aplikacija → identity-service (prijava)	OpenID Connect (*Authorization Code* + PKCE)	standardni tok za javne odjemalce; žeton dostopa velja za vse API-je
prehod in storitve → identity-service	JWKS; OAuth 2.0 *client credentials*	preverjanje žetonov brez klica na zahtevo; žetoni za klice med storitvami in iz funkcij
aplikacija → seznam pogovorov	GraphQL (conversation-service)	en klic za gnezdeno sliko: člani, vloge, zadnja aktivnost
strežnik → aplikacija (nova sporočila, tipkanje, prisotnost)	WebSocket (WSS)	potiskanje z nizko zakasnitvijo, brez anketiranja
message-service → conversation-service (članstvo pri pošiljanju)	gRPC	klicatelj potrebuje odgovor; nizka zakasnitev, pogodba Protobuf; prekinjevalec toka in predpomnilnik
conversation-service → identity-service (razrešitev uporabnikov in naprav)	gRPC	preverjanje, da je novi član v isti organizaciji
dejstva (sporočilo shranjeno, član odstranjen, naprava preklicana …)	NATS JetStream, CloudEvents	več porabnikov, šibka sklopljenost, ponovno branje; idempotentni porabniki
notification-service → FCM → naprava	HTTPS + OAuth 2.0	zunanji API za potisna obvestila brez vsebine
aplikacija ↔ Blob Storage; Blob → Function → media-service	HTTPS z URL SAS; Event Grid; HTTPS prek prehoda (OAuth 2.0)	veliki šifrirani bloki ne obremenjujejo gruče (vzorec *valet key*)

Dogodek (subjekt NATS)	Proizvajalec	Porabniki	Namen
`identity.device.registered`	identity	keys, notification, realtime	lokalna kopija naprav
`identity.device.revoked`	identity	keys, notification, realtime	izbris ključev in žetonov, prekinitev povezav
`identity.organization.created`	identity	conversation, media, notification	priprava najemnika (kvote, pravila hrambe)
`keys.identity-key.changed`	keys	realtime	opozorilo stikom o spremenjeni varnostni številki
`keys.key-packages.low`	keys	notification	tiho obvestilo napravi, naj naloži nove pakete ključev
`conversations.member-added`	conversation	message, media, realtime, notification	lokalne kopije članstva za avtorizacijo in usmerjanje
`conversations.member-removed`	conversation	message, media, realtime, notification	odvzem dostopa do pogovora in priponk
`messages.stored`	message	realtime, notification, conversation	dostava v živo, potisno obvestilo, zadnja aktivnost
`media.attachment.available`	media	realtime	obvestilo napravam pošiljatelja
`media.attachment.expired`	media	realtime	brisanje lokalnih kopij
`realtime.presence.changed`	realtime	notification	lokalna kopija prisotnosti (komu poslati potisno obvestilo)

Vsak dogodek je CloudEvent s tipom, ki vsebuje različico (npr. `si.sepet.messages.stored.v1`), z `id` za idempotentne porabnike in `correlationid` za sledenje; kanali so opisani v AsyncAPI 3.0.

5 Mikrostoritve in njihove funkcionalnosti

Vse storitve so napisane v .NET 10 po načelih Clean Architecture (sloji *Domain*, *Application*, *Infrastructure*, *Api*), imajo končne točke `/health/live`, `/health/ready` in `/health/startup`, preverjajo žetone JWT prek JWKS, izvažajo metrike in dnevnike prek OpenTelemetry ter se namestijo s svojim Helm chartom.

identity-service — ponudnik identitet (OAuth 2.0, OpenID Connect) in računi

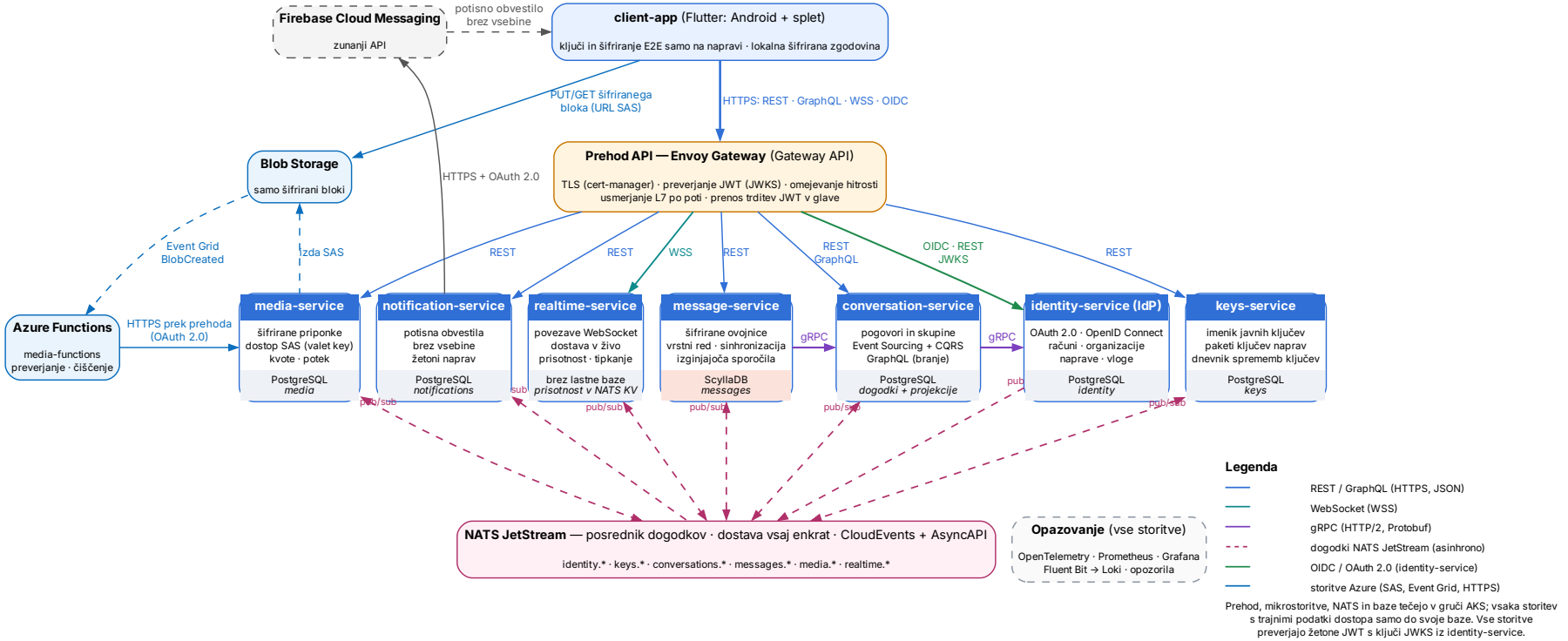
Lastništvo: Tibor Koderman · PostgreSQL `identity`. Identiteta je običajno generična poddomena, tu pa je del jedra: v arhitekturi MLS ima vlogo storitve za avtentikacijo, ki veže uporabnika na podpisne ključe njegovih naprav. Protokol in hrambo gesel prepustimo preverjenima knjižnicama (OpenIddict, ASP.NET Core Identity).

- OpenID Connect in OAuth 2.0: *Authorization Code* + PKCE za aplikacijo, *client credentials* za storitve in funkcije, osveževalni žetoni z rotacijo; discovery, JWKS in `userinfo`.
- Računi: registracija, prijava, TOTP kot drugi faktor, zaklepanje po zaporednih neuspelih prijavah.
- Organizacije (najemniki), povabila in vloge (`member`, `tenant-admin`, `platform-admin`); trditev `tenant_id` v žetonu.
- Naprave: registracija, pregled in preklic; podpis poverilnice naprave (MLS *credential*).
- Vmesniki: OIDC `/connect/*`, REST `/v1/me`, `/v1/users`, `/v1/devices`, `/v1/organizations/{id}/…`; gRPC strežnik `UserDirectory`; dogodki `identity.*`.

keys-service — imenik javnih ključev

Lastništvo: Tibor Koderman · PostgreSQL `keys`. Skupaj z message-service izvaja vlogo dostavne storitve iz arhitekture MLS (RFC 9750): hrani in razdeljuje pakete ključev.

- Objava identitetnega ključa naprave in zaloge paketov ključev (KeyPackage), s preverjanjem podpisa ob nalaganju.
- Atomarni prevzem paketa ključev za vsako napravo novega člana — vsak paket se porabi natanko enkrat (`FOR UPDATE SKIP LOCKED`).
- Dnevnik sprememb ključev, verižen z zgoščenimi vrednostmi, za varnostne številke in opozorila ob spremembi ključa.



Slika 1: Začetna shema arhitekture aplikacije Šepet s komunikacijskimi protokoli. Vsaka mikrostoritev s trajnimi podatki ima lastno podatkovno bazo (spodnji pas okvirja); do tujih podatkov dostopa le prek API-jev ali dogodkov.

- Opozorilo, ko napravi zmanjkuje paketov ključev; ob preklicu naprave izbris njenih ključev.
- Vmesniki: REST `/v1/devices/{id}/key-packages`, `/v1/key-packages/claim`, `/v1/key-log`; dogodki `keys.*`; porablja `identity.*`.

conversation-service — pogovori in skupine (Event Sourcing + CQRS)
Lastništvo: Amar Mehić · PostgreSQL `conversations` (Marten). Upravlja pogovore ena na ena in skupine; stanje je zaporedje dogodkov.

- Pisalni model (agregat `Conversation`): ustvari pogovor ali skupino, dodaj ali odstrani člana, spremeni vlogo, nastavi čas izginjanja sporočil, posodobi šifrirane nastavitve (ime in slika skupine sta šifrirana).
- Shramba dogodkov: `ConversationCreated`, `MemberAdded`, `MemberRemoved`, `RoleChanged`, `SettingsChanged` …; integracijski dogodki se objavijo iz nje, zato ni problema dvojnega zapisa.
- Bralni modeli (projekcije): seznam pogovorov uporabnika z zadnjo aktivnostjo in članstvo; GraphQL (Hot Chocolate) za aplikacijo, gRPC strežnik `Membership` za druge storitve.
- Ponovno predvajanje dogodkov obnovi projekcije iz nič (demonstracija zahteve 12).
- Vmesniki: REST `/v1/conversations`, `/v1/conversations/{id}/members`; GraphQL `/graphql`; gRPC `Membership`; dogodki `conversations.*`; porablja `messages.stored` in `identity.organization.created`.

message-service — dostava šifriranih sporočil
Lastništvo: Amar Mehić · ScyllaDB `messages`. Dostavna storitev v smislu arhitekture MLS: hrani in ureja šifrirane ovojnice, ne da bi jih lahko prebrala.

- Sprejem šifrirane ovojnice (REST z `Idempotency-Key`), preverjanje članstva pošiljatelja prek gRPC in dodelitev časovno urejenega identifikatorja (timeuuid).
- Urejanje sprememb skupine (MLS Commit): za vsako epoho sprejme natanko eno spremembo s pogojnim zapisom (*lightweight transaction*), zato vsi člani uporabijo spremembe v istem vrstnem redu.
- Hramba: particija (najemnik, pogovor, časovno vedro), vrstice urejene po identifikatorju; TTL za izginjajoča sporočila.
- Sinhronizacija naprave po ponovni vzpostavitvi povezave (`GET …/messages?after={id}`) in kazalci prebranega po napravi.
- Vmesniki: REST `/v1/conversations/{id}/messages`, `/v1/conversations/{id}/commits`; gRPC odjemalec `Membership`; dogodek `messages.stored`; porablja `conversations.*`.

realtime-service — dostava v živo
Lastništvo: Amar Mehić · brez lastne baze (prisotnost v NATS KV s časom poteka). Skalira se po številu povezav, ločeno od message-service.

- Povezave WebSocket, overjene z žetonom JWT; dostava novih ovojnic, sprememb pogovorov in opozoril o ključih povezanim napravam.
- Prisotnost in indikator tipkanja (efemerno); spremembe prisotnosti objavi kot dogodek.
- Vmesniki: WebSocket `/v1/ws`; dogodek `realtime.presence.changed`; porablja `messages.stored`, `conversations.*`, `keys.identity-key.changed`, `identity.device.revoked` (prekine povezave preklicane naprave) in `media.attachment.*`.

media-service — šifrirane priponke
Lastništvo: Eva Vidic · PostgreSQL `media` + Azure Blob Storage.

- Izda kratkotrajen URL SAS za nalaganje ali prenos (vzorec *valet key*); bloki potujejo neposredno v Blob Storage, mimo gruče.
- Metapodatki priponke (velikost, zgoščena vrednost šifriranega bloka, stanje, čas poteka) in kvote po organizaciji.
- Prenos dovoli le članom pogovora (lokalna kopija članstva iz dogodkov).
- Vmesniki: REST `/v1/attachments`, `/v1/attachments/{id}`; notranji webhook za Azure Function; dogodki `media.*`; porablja `conversations.*` in `identity.organization.created`.

media-functions — brezstrežniški funkciji (Azure Functions)
Lastništvo: Eva Vidic · .NET 10 isolated, Flex Consumption, Application Insights.

- `AttachmentValidator` (Event Grid `BlobCreated`): preveri velikost in zavrne bloke, ki so videti kot nešifrirane slike (prepoznavni začetni bajti formatov JPEG, PNG …); rezultat sporoči media-

service prek prehoda API (OAuth 2.0 *client credentials*).
- `AttachmentJanitor` (časovni sprožilec): briše potekle bloke in sporoči izbrisane priponke.

notification-service — potisna obvestila

Lastništvo: Eva Vidic · PostgreSQL `notifications`.

- Registracija žetonov FCM po napravah in nastavitve obveščanja (utišani pogovori).
- Ob `messages.stored` napravam brez povezave pošlje potisno obvestilo **brez vsebine** prek FCM HTTP v1 (OAuth 2.0 s storitvenim računom); aplikacija se zbudi in sama prenese šifrirane ovojnice.
- Ponovni poskusi, prekinjevalec toka in omejevanje hitrosti proti FCM; neveljavne žetone odstrani.
- Vmesniki: REST `/v1/devices/{id}/push-token`, `/v1/preferences`; porablja `messages.stored`, `conversations.*`, `realtime.presence.changed`, `keys.key-packages.low` in `identity.device.*`.

client-app — uporabniški vmesnik (Flutter)

Lastništvo: Eva Vidic; zaslone za svoje storitve prispeva vsakdo v ekipi.

- Podstrani: prijava, seznam pogovorov, pogovor (oblikovano besedilo, slike), nova skupina in člani, profil z napravami in varnostno številko (koda QR), nastavitve, skrbništvo organizacije (samo `tenant-admin`).
- Prijava z OpenID Connect (*Authorization Code* + PKCE), komunikacija z REST, GraphQL in WebSocket; zgodovina v šifrirani lokalni zbirki, ključi v varni shrambi naprave (Android Keystore; na spletu neizvozljiv ključ WebCrypto v IndexedDB, kar je šibkejša zaščita).
- Oblikovano besedilo kot omejen Markdown (krepko, ležeče, prečrtano, koda, seznami, citati, povezave, omembe), ki ga upodobi odjemalec; strežnik ga vidi le šifriranega.

6 Primeri uporabe

Oznaka	Primer uporabe	Raven	Sodelujoče storitve
PU-01	Registracija in prijava (OIDC + PKCE), registracija naprave in objava javnih ključev	osnovni	identity, keys
PU-02	Iskanje uporabnika v organizaciji in začetek pogovora ena na ena	osnovni	identity, keys, conversation, message
PU-03	Pošiljanje in prejemanje šifriranega sporočila — v živo in po ponovni povezavi	osnovni	message, conversation, realtime
PU-04	Pisanje oblikovanega besedila (krepko, ležeče, koda, seznami, povezave, omembe)	osnovni	aplikacija, message
PU-05	Seznam pogovorov z zadnjo aktivnostjo (GraphQL); neprebrano izračuna naprava	osnovni	conversation, message
PU-06	Potisno obvestilo brez vsebine za napravo brez povezave	osnovni	message, realtime, notification, FCM
PU-07	**Pošiljanje šifrirane slike z opisom v skupinski pogovor (kompleksni primer)**	napredni	media, Functions, conversation, message, realtime, notification
PU-08	Ustvarjanje skupine ter dodajanje in odstranjevanje članov (nova epoha ključev)	napredni	conversation, identity, keys, message, realtime
PU-09	Preverjanje varnostne številke s kodo QR in opozorilo ob spremembi ključa stika	napredni	keys, realtime
PU-10	Izginjajoča sporočila in priponke s časom poteka	napredni	conversation, message, media, Functions
PU-11	Skrbništvo organizacije: povabila, dodelitev vloge `tenant-admin`, pravila hrambe	napredni	identity
PU-12	Pregled in preklic izgubljene naprave (izbris ključev, žetonov in povezav)	napredni	identity, keys, notification, realtime
PU-13	Obnova bralnih modelov s ponovnim predvajanjem dogodkov (operater)	napredni	conversation
PU-14	Delovanje ob izpadu posamezne storitve (npr. media ali notification)	napredni	vse

PU-07 — pošiljanje šifrirane slike z opisom v skupinski pogovor. *Akterji:* Ana (pošiljateljica), Bor (povezan), Cene (brez povezave), vsi člani skupine v organizaciji FRI. *Predpogoj:* Ana je prijavljena in ima veljaven žeton; skupina je v epohi MLS *n*.

1. **Aplikacija (Ana):** Ana izbere sliko in doda opis. Aplikacija odstrani metapodatke EXIF, izdela sličico, ustvari naključni ključ *K* in sliko šifrira (AES-256-GCM); izračuna SHA-256 šifriranega bloka.
2. **Prehod API → media-service (REST):** prehod preveri JWT in omejitev hitrosti; media-service preveri kvoto organizacije in Anino članstvo, shrani metapodatke (stanje `pending`) in vrne URL SAS za nalaganje (samo zapis, 10 minut, omejena velikost).
3. **Aplikacija → Blob Storage:** šifrirani blok naloži neposredno, mimo gruče.
4. **Blob Storage → Event Grid → Azure Function:** `AttachmentValidator` preveri velikost in zavrne bloke, ki so videti kot nešifrirane slike, nato pokliče media-service (OAuth 2.0); ta priponko označi kot `available` in objavi `media.attachment.available`.
5. **Aplikacija → message-service (REST):** sestavi sporočilo {id priponke, *K*, SHA-256, tip, mere, opis v Markdownu}, ga šifrira s skupinskim ključem epohe *n* in pošlje z `Idempotency-Key`.
6. **message-service → conversation-service (gRPC):** preveri, ali je Anina naprava član skupine v epohi *n* (časovna omejitev, ponovni poskus, prekinjevalec toka; ob izpadu kratkotrajen predpomnilnik članstva).
7. **message-service → ScyllaDB, NATS:** shrani ovojnico s časovno urejenim identifikatorjem (in TTL, če skupina uporablja izginjajoča sporočila) ter objavi `messages.stored`.
8. **realtime-service → Bor (WebSocket):** dostavi ovojnico; Borova aplikacija sporočilo dešifrira, prek media-service dobi URL SAS za branje, prenese blok, preveri SHA-256, ga dešifrira s *K* in prikaže.
9. **notification-service → FCM → Cene:** po lokalni kopiji prisotnosti (dogodki realtime-service) Cene ni povezan, zato dobi potisno obvestilo brez vsebine; aplikacija se zbudi, prenese manjkajoče ovojnice (`?after={id}`) in prikaže lokalno obvestilo.

Odpornost: če notification-service ne deluje, se sporočilo vseeno shrani in dostavi povezanim napravam, obvestila pa se pošljejo, ko se storitev vrne (trajni porabnik JetStream). Če ne deluje media-service, besedilna sporočila delujejo naprej, nalaganje slike pa se ponovi z eksponentnim odlogom. Če ne deluje FCM, se v notification-service odpre prekinjevalec toka.

7 Pokritost zahtev predmeta

Ciljamo na vseh 20 osnovnih zahtev (100 točk) in vse bonusne zahteve; bonusne točke so rezerva za morebitne izgube drugje.

#	Zahteva	Točke	Načrtovana izvedba
1	Repozitorij	2	skupina GitLab `fri-mag/rso` z javnimi projekti, README, veji `main` in `dev`, zaščitena `main`
2	Mikrostoritve, cloud-native	5	7 mikrostoritev + GUI, Clean Architecture, 12-faktorska načela, manifesti Kubernetes prek Helma
3	Baza na storitev	3	5 × PostgreSQL, ScyllaDB, ločene poverilnice, migracije in začetni podatki
4	Dokumentacija	3	projekt `docs`: model C4, zapisi ADR, navodila za razvijalce, tehnično poročilo
5	Dokumentacija API	4	OpenAPI 3.1 (Scalar) s primeri in napakami RFC 9457; AsyncAPI, `.proto`, GraphQL SDL
6	CI/CD	6	GitLab CI: testi → slika v registru → samodejna namestitev v *dev* (veja `dev`) in *prod* (veja `main`, z odobritvijo)
7	Helm	5	chart za vsako storitev; vrednosti `values-dev` in `values-prod` (+ lokalni `values-local`)
8	Namestitev v oblak	6	Azure AKS, javni dostop prek prehoda z domeno in TLS
9	Zunanji API	4	FCM HTTP v1 z OAuth 2.0 v notification-service
10	Večnajemniškost	6	organizacije v identity-service, trditev `tenant_id` v JWT; RLS v PostgreSQL in najemnik v ključu particije ScyllaDB; ločen imenski prostor in baze za izoliranega najemnika
11	Preverjanje zdravja	5	`/health/live`, `/health/ready`, `/health/startup` + sonde Kubernetes

#	Zahteva	Točke	Načrtovana izvedba
12	Event Sourcing in CQRS	6	conversation-service: shramba dogodkov, projekcije, bralni model GraphQL, ponovno predvajanje
13	Izolacija in odpornost	6	časovne omejitve, ponovni poskusi, prekinjevalec toka, pregrade; demonstracija izpadov
14	Upravljanje konfiguracije	5	datoteke → okoljske spremenljivke → ConfigMap z osvežitvijo brez ponovne namestitve → Secrets in Key Vault → Azure App Configuration (zastavice funkcionalnosti)
15	Grafični vmesnik	5	Flutter (Android + splet), več kot dve podstrani, REST
16	Brezstrežniška funkcija	6	Azure Functions: Event Grid `BlobCreated` in časovni sprožilec, Application Insights, skaliranje Flex Consumption
17	GraphQL in gRPC	5	GraphQL v conversation-service; gRPC message → conversation in conversation → identity
18	Pretakanje dogodkov	6	NATS JetStream; vseh 7 storitev objavlja ali porablja dogodke
19	Centralizirani dnevniki	6	Fluent Bit → Loki → Grafana; opozorila na vzorce (npr. niz neuspelih prijav, napake podpisov)
20	Metrike	6	OpenTelemetry → Prometheus → Grafana; RED/USE; obremenitveni testi k6 in optimizacija
21	Vmesna predstavitev (bonus)	+10	14.–18. 12. 2026
22	Terraform (bonus)	+3	moduli za AKS, Storage, Functions, Key Vault in opazovanje; `plan` in `apply` v CI
23	Prehod API (bonus)	+8	Envoy Gateway: usmerjanje L7, TLS, JWT, omejevanje hitrosti, prenos trditev v glave
24	IAM, OAuth2, OIDC (bonus)	+4	lasten ponudnik identitet: OIDC + PKCE, obsegi OAuth 2.0 za API-je, vloge in dodeljevanje pravic na skrbniški strani

8 Časovni načrt in tveganja

Obdobje	Mejnik
do 26. 10.	prijava teme, skupina in projekti GitLab, ključne odločitve
27. 10.–8. 11.	»hodeče okostje«: predloga storitve, GitLab CI, Helm, AKS s Terraformom, ponudnik identitet, prijava v aplikaciji; preizkus integracije MLS v Flutter
9.–22. 11.	jedro: ključi in šifriranje 1:1, message-service s ScyllaDB, realtime-service, NATS (1. kolokvij 17. 11.)
23. 11.–13. 12.	skupine (Event Sourcing, CQRS, GraphQL, gRPC), slike in Azure Functions, obvestila FCM, prehod API, opazovanje
14.–18. 12.	vmesna predstavitev
19. 12.–3. 1.	večnajemniškost, odpornost, upravljanje konfiguracije, metrike in optimizacija
4.–11. 1. 2027	tehnično poročilo in priprava predstavitve; oddaja 11. 1. 2027

- **Integracija MLS v Flutter (zlasti splet):** časovno omejen preizkus v prvem tednu; ker zaledje hrani le neprozorne pakete ključev in ovojnice, lahko z manjšimi spremembami dveh storitev preklopimo na Signal Protocol.
- **Lasten ponudnik identitet:** varnostno kritična koda → samo preverjeni knjižnici (OpenIddict, ASP.NET Core Identity), integracijski testi tokov OIDC in zaklepanje po neuspelih prijavah.
- **Proračun Azure (100 USD na študenta):** gručo ustavimo, ko je ne uporabljamo, razvijamo lokalno (Docker Compose, kind), uporabljamo majhna vozlišča in brezplačne ali najcenejše stopnje storitev.
- **Obseg (7 storitev):** skupna predloga storitve in skupne predloge GitLab CI; vsakdo v ekipi skrbi za svoje storitve.